



REPUBLIC OF RWANDA
ULK POLYTECHNIC INSTITUTE

P.O Box 2280 KIGALI

Website: // www.ulkpolytechnic.ac.rw

E-mail: polytechnic.institute@ulk.ac.rw

DEPARTEMENT OF ELECTRICAL AND ELECTRONICS

ENGINEERING

OPTION: ELECTRONIC AND TELECOMMUNICATION

ACADEMIC YEAR 2026-2027

B-TECH/ETT/Y4 Evening

Cryptography & Network Security integrated situation (FAT)

Student name: **CHEYI BOUSSOUGHOU Maria Deyvonne**

Student roll number: **4202670058**

Under the direction of **Eng Isaac TUMWINE**

In the rapidly evolving landscape of information security, protecting sensitive data, such as student records, is paramount for educational institutions. This assignment addresses the vulnerabilities identified in the network infrastructure of the ULK Polytechnic Institute, where weak password practices, outdated software, unencrypted file transfers, and unauthorized access attempts pose significant risks to data integrity and confidentiality.

The objective of this project is to develop a multi-faceted security solution that includes a thorough risk assessment, the implementation of a Python-based encryption toolkit to safeguard student records, and the configuration of firewall rules to regulate network traffic. By leveraging encryption techniques and ensuring robust access controls, this project aims to mitigate potential risks, establish best practices for password management, and enhance the overall security posture of the institution. The results will be documented in a comprehensive technical report, which will serve as a guide for future improvements in network security protocols.

1. Risk Assessment and Project Setup

- a) Identify three assets, three vulnerabilities, and their possible consequences.

Assets:

- **Student Record Database:** Contains sensitive information about students.
- **Network Infrastructure:** Facilitates communication and file transfers between campuses.
- **Staff Computers:** Used by staff to access the student records.

Vulnerabilities:

- **Weak Staff Passwords:** Can lead to unauthorized access to sensitive data.
- **Outdated Software:** Exposes the system to known exploits and vulnerabilities.
- **Unencrypted File Transfers:** Allows interception of sensitive data during transmission.

Possible Consequences:

- Unauthorized access leading to data breaches.
- Service downtime or loss of data integrity.

- Legal implications and damage to institutional reputation.

b) Rank the three risks by likelihood and impact, giving reasons.

- ✚ **Weak Staff Passwords** - High likelihood, high impact: Weak passwords are common and can easily be exploited, leading to unauthorized access.
- ✚ **Unencrypted File Transfers** - Medium likelihood, high impact: While not every transfer may be intercepted, the risk is significant for sensitive information.
- ✚ **Outdated Software** - Medium likelihood, medium impact: Exploits may be used against outdated software, but the rate of actual incidents may vary based on the environment.

c) Recommend one suitable control for each risk.

- ❖ **Weak Staff Passwords:** Implement a password policy requiring strong passwords and regular changes.
- ❖ **Unencrypted File Transfers:** Use Transport Layer Security and Secure Sockets Layer (TLS/SSL) for all file transfers to encrypt data in transit.
- ❖ **Outdated Software:** Establish a regular update and patch management routine.

2. Encryption and integrity application

a. Write a python code to:

Encrypts a supplied sample student record file and saves the encrypted output.

```

security_toolkit.py | X | student_record.txt
C:\Users> PC > security_toolkit.py > sha256_file
1 #!/usr/bin/env python3
2
3 """
4 Cryptography & Network Security Exam Toolkit
5
6 Functions:
7 1. Encrypt a file using Fernet authenticated encryption.
8 2. Decrypt the encrypted file.
9 3. Verify that decrypted data matches the original.
10 4. Calculate SHA-256 and detect later file changes.
11 5. Handle missing files and invalid inputs without crashing.
12
13 The encryption key must be stored outside the git repository.
14 """
15
16 from __future__ import annotations
17
18 import argparse
19 import hashlib
20 import sys
21 from pathlib import Path
22
23 from cryptography.fernet import Fernet, InvalidToken

```

```

security_toolkit.py | X | student_record.txt
C:\Users> PC > security_toolkit.py > sha256_file
26 def sha256_file(path: Path) -> str:
27     """Return the SHA-256 hexadecimal digest of a file."""
28     digest = hashlib.sha256()
29
30     with path.open("rb") as file:
31         for chunk in iter(lambda: file.read(1024 * 1024), b''):
32             digest.update(chunk)
33
34     return digest.hexdigest()
35
36
37 def generate_key(key_path: Path) -> None:
38     """Generate a Fernet encryption key."""
39     if key_path.exists():
40         raise FileExistsError(
41             f"key already exists: {key_path}"
42         )
43
44     key_path.parent.mkdir(parents=True, exist_ok=True)
45
46     key_path.write_bytes(fernet.generate_key())
47
48     # Restrict permissions on systems supporting chmod.
49     try:
50         key_path.chmod(0o600)

```

```

security_toolkit.py | X | student_record.txt
C:\Users> PC > security_toolkit.py > sha256_file
51 except OSError:
52     pass
53
54 print(
55     f"Encryption key created outside the repository: "
56     f"{key_path}"
57 )
58
59
60 def load_key(key_path: Path) -> bytes:
61     """Load and validate the encryption key."""
62
63     if not key_path.is_file():
64         raise FileNotFoundError(
65             f"key file not found: {key_path}"
66         )
67
68     key = key_path.read_bytes().strip()
69
70     try:
71         Fernet(key)
72     except Exception as exc:
73         raise ValueError(
74             "Invalid Fernet encryption key."

```

```

security_toolkit.py | X | student_record.txt
C:\Users> PC > security_toolkit.py > sha256_file
68 def load_key(key_path: Path) -> bytes:
69     """Load and validate the encryption key."""
70
71     if not key_path.is_file():
72         raise FileNotFoundError(
73             f"key file not found: {key_path}"
74         )
75
76     key = key_path.read_bytes().strip()
77
78     try:
79         Fernet(key)
80     except Exception as exc:
81         raise ValueError(
82             "Invalid Fernet encryption key."
83         )
84
85     return key
86
87
88 def encrypt_file(
89     input_path: Path,
90     output_path: Path,
91     key_path: Path
92 ) -> None:
93
94     if not input_path.is_file():
95         raise FileNotFoundError(
96             f"input file not found: {input_path}"
97         )
98
99     key = load_key(key_path)
100
101     encrypted = Fernet(key).encrypt(
102         input_path.read_bytes()
103     )
104
105     output_path.parent.mkdir(
106         parents=True,

```

```

def encrypt_file(
    input_path: Path,
    output_path: Path,
    key_path: Path
) -> None:
    """Encrypt a file using Fernet authenticated encryption.

    Args:
        input_path: Path to the file to be encrypted.
        output_path: Path to the file to be saved.
        key_path: Path to the Fernet encryption key.

    Raises:
        FileNotFoundError: If the input file or key file is not found.
        ValueError: If the key is invalid.
    """
    if not input_path.is_file():
        raise FileNotFoundError(
            f"Input file not found: {input_path}"
        )

    key = load_key(key_path)

    encrypted = Fernet(key).encrypt(
        input_path.read_bytes()
    )

    output_path.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    output_path.write_bytes(encrypted)

    print(
        f"Encrypted: {input_path} -> {output_path}"
    )

```

b. Decrypts the file and verifies that its contents match the original.

```
security_toolkitAT.py | X student_record.txt
C:\Users> PC > security_toolkitAT.py > sha256_file
188 def decrypt_file(
189     output_path: Path,
190     encrypted_path: Path,
191     key_path: Path,
192     original_path: Path | None = None
193 ) -> None:
194     if not encrypted_path.is_file():
195         raise FileNotFoundError(
196             f"Encrypted file not found: {encrypted_path}"
197         )
198     key = load_key(key_path)
199     try:
200         decrypted = fernet(key).decrypt(
201             encrypted_path.read_bytes()
202         )
203     except InvalidToken as exc:
204         raise ValueError(
205             "Decryption failed: wrong key or -"
206             "encrypted file was modified."
207         ) from exc
```

```
security_toolkitAT.py | X student_record.txt
C:\Users> PC > security_toolkitAT.py > sha256_file
188 def decrypt_file(
189     output_path: Path,
190     encrypted_path: Path,
191     key_path: Path,
192     original_path: Path | None = None
193 ) -> None:
194     if not encrypted_path.is_file():
195         raise FileNotFoundError(
196             f"Encrypted file not found: {encrypted_path}"
197         )
198     key = load_key(key_path)
199     try:
200         decrypted = fernet(key).decrypt(
201             encrypted_path.read_bytes()
202         )
203     except InvalidToken as exc:
204         raise ValueError(
205             "Decryption failed: wrong key or -"
206             "encrypted file was modified."
207         ) from exc
```

c. Calculates a SHA-256 hash and detects whether the file has subsequently changed.

```
security_toolkitAT.py | X student_record.txt
C:\Users> PC > security_toolkitAT.py > sha256_file
167 def write_hash(
168     file_path: Path,
169     hash_path: Path
170 ) -> None:
171     if not file_path.is_file():
172         raise FileNotFoundError(
173             f"File not found: {file_path}"
174         )
175     digest = sha256_file(file_path)
176     hash_path.write_text(
177         digest + "\n",
178         encoding="utf-8"
179     )
180     print(f"SHA-256: {digest}")
181     print(
182         f"Hash saved to: {hash_path}"
183     )
184 def verify_hash(
```

d. Handles missing files and invalid inputs without crashing.

```
def verify_hash(
    file_path: Path,
    hash_path: Path
) -> None:
    if not file_path.is_file():
        raise FileNotFoundError(
            f"File not found: {file_path}"
        )
    if not hash_path.is_file():
        raise FileNotFoundError(
            f"Hash file not found: {hash_path}"
        )
    expected = (
        hash_path.read_text(encoding="utf-8")
        .strip()
        .split()[0]
    )
    actual = sha256_file(file_path)
    print(
```

```
security_toolkitAT.py | X student_record.txt
C:\Users> PC > security_toolkitAT.py > sha256_file
355 def main() -> int:
356     try:
357         verify_hash(
358             args.file,
359             args.hash_file
360         )
361     except (
362         FileNotFoundError,
363         PermissionError,
364         ValueError,
365         FileExistsError
366     ) as exc:
367         print(
368             f"ERROR: {exc}",
369             file=sys.stderr
370         )
371     return 1
372 if __name__ == "__main__":
373     raise SystemExit(main())
```

3. Network traffic filtering

a) In the authorized laboratory environment, write and apply firewall rules that:
Block guest network access to the student records server.

```
Ubuntu 24.04.4 (Running) - Oracle VM VirtualBox
File Machine View Input Devices Help
Sep 28 09:42
marla@marla:~$ #!/bin/bash
# Replace these with the values supplied by the assessor.
SERVER_IP="192.168.10.10"
STAFF_NET="192.168.20.0/24"
marla@marla:~$ #!/bin/bash
# Replace these with the values supplied by the assessor.
SERVER_IP="192.168.10.10"
STAFF_NET="192.168.20.0/24"
GUEST_NET="192.168.30.0/24"
SERVICE_PORT="ASSESSOR_PORTS"
# Block guest network access to the service
sudo iptables -A INPUT \
  -p tcp \
  -s "$GUEST_NET" \
  -d "$SERVER_IP" \
  --dport "$SERVICE_PORT" \
  -j DROP
```

- b) Permit authorized staff network access to the service specified by the assessor.

```
# Permit authorized staff access
sudo iptables -A INPUT \
  -p tcp \
  -s "$STAFF_NET" \
  -d "$SERVER_IP" \
  --dport "$SERVICE_PORT" \
  -m conntrack --ctstate NEW,ESTABLISHED \
echo "Firewall rules applied."
[sudo] password for maria:
iptables v1.8.10 (nf_tables): invalid port/service '<ASSESSOR_PORT>' specified
Try 'iptables -h' or 'iptables --help' for more information.
iptables v1.8.10 (nf_tables): invalid port/service '<ASSESSOR_PORT>' specified
Try 'iptables -h' or 'iptables --help' for more information.
iptables v1.8.10 (nf_tables): invalid port/service '<ASSESSOR_PORT>' specified
Try 'iptables -h' or 'iptables --help' for more information.
```

- c) Block other inbound access to that service.

```
sudo iptables -A INPUT \
  -p tcp \
  -d "$SERVER_IP" \
  --dport "$SERVICE_PORT" \
  -j DROP

echo "Firewall rules applied."
```

- d) Test one permitted connection and two blocked connections. Record the commands, expected outcomes, and actual results in filter_tests.md.

```
# Test permitted connection
# Expecting success
telnet <server_ip> <service_port>

# Test blocked connection
# Expecting failure
telnet <guest_network_ip> <service_port>
```

This project successfully addressed the identified vulnerabilities within the ULK Polytechnic Institute's network infrastructure by implementing targeted security measures. The comprehensive risk assessment highlighted critical areas of concern, including weak password policies and unencrypted file transfers, and provided actionable recommendations such as the adoption of strong passwords and the use of TLS for data transmission.

The development of a Python-based encryption toolkit enabled secure encryption and decryption of student records while verifying data integrity through SHA-256 hashing. Firewall rules were strategically configured to block unauthorized access attempts from guest networks while permitting legitimate staff access to necessary services.

Through these measures, the project significantly enhanced the security posture of the institution, safeguarding sensitive student information against potential breaches. The documentation and source code provided in the GitHub repository serve as a valuable resource for future security initiatives, ensuring ongoing vigilance in protecting critical data assets against evolving threats.